USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA
DEPARTAMENTO DE ELECTRÓNICA E COMPUTACIÓN

Assignment 7: Running a Hadoop cluster with Docker containers

Artificial Intelligence in Big Data Environments

In this practical, we will use Docker containers to deploy a Hadoop cluster. The practical consists of two parts. In the first part, we will configure a Hadoop cluster using Docker containers. This part is guided. You only need to follow the instructions. In the second part, you have to complete four tasks on your own. The tasks are described in the second section of this document.

Instructions to submit this mandatory practical

This practical is **mandatory and must be completed in pairs**. You must submit a report containing the answers to the tasks in Section 2, along with your `Dockerfile` and `docker-compose.yml` files. **All files must be compressed into a single zip archive named *Assignment7_Surname1_Name1_Surname2_Name2.zip***. For example, if John Doe and Jane Luther complete the practical together, the file must be named *Assignment7_Doe_John_Luther_Jane.zip*.

The answers have to demonstrate that you have completed and understood the proposed tasks. The report should include at least the following:

1. Commands used to complete the tasks.
2. Screenshots showing that the Backup and TimeLineServer nodes have been created and are running correctly.
3. Screenshots showing two new nodes were added and one node was decommissioned.
4. Screenshots showing the nodes separated by racks.
5. If you use AI tools, you must include an "AI Tool Usage" section (see the end of this document for a template).
6. The Dockerfile used to create the base image.
7. The last version of the docker-compose.yml.

More details about the required content of the report are given at the end of each task.

The deadline to submit this practical ends on **Sunday, 22 November at 23:59**.

Index

1 Installation, setup, and deployment of a Hadoop cluster **3**

1.1 NameNode/ResourceManager 5

1.2 DataNode/NodeManager 13

1.3 Deployment of the Hadoop cluster using Docker Compose 17

1.4 Basic use of the HDFS filesystem 20

1.5 Test a simple MapReduce job 22

2 Assignment **23**

2.1 Task 1: Add a Backup node and a TimeLineServer to the cluster 23

2.2 Task 2: Decommissioning a DataNode/NodeManager 26

2.3 Task 3: Adding a new DataNode/NodeManager 27

2.4 Task 4: Rack Awareness 28

Requirements

As this practical is done in pairs, it is recommended to choose the most powerful computer of the two. Your PC should fulfill the following requirements:

- At least 8GB of RAM (preferably 16GB).
- Preferably, not a computer with an ARM processor (M1, M2, etc).
- Preferably, Linux operating system.
  - In case you use Windows, version 10 or higher is recommended with WSL2 or Hyper-V enabled.

Moreover, you need to install Docker Engine and Docker Compose on your machine. You can find the installation instructions for your operating system at `https://docs.docker.com/engine/installation/` or install Docker Desktop, which includes Docker Engine and Docker Compose, from `https://docs.docker.com/desktop/`.

1 Installation, setup, and deployment of a Hadoop cluster

Following the current industry standards, we will deploy a Hadoop cluster using Docker containers. We start by creating a Docker image that will serve as a base for creating other images. The image will be based on the latest Ubuntu LTS and will have Hadoop (without any configuration) and its dependencies (Java, maven, python3, etc.) installed. In addition, the image will have two defined users:

- **studentName1StudentName2**: administrator user with permissions to configure/run/terminate Hadoop daemons. Change *studentName1StudentName2* to your names. For example, if your names are John and Jane, the user will be *johnjane*.
- **luser** (password *luser*): non-privileged user that will execute MapReduce applications on the cluster.

As in all Linux distributions, we also have a *root* user. In case we need to execute commands as root, it will be indicated with a `#` at the beginning of the command. Otherwise (`$`), we will run the commands as *studentName1StudentName2* or *luser* (it will be indicated in the text).

With the given information, complete the following *Dockerfile* to create the base image:

```
# Use the latest Ubuntu LTS as the base image
COMPLETE

# COMPLETE with your studentName1StudentName2
ENV USERNAME=COMPLETE

# Directory where we will install Hadoop
ENV BASE_DIR=/opt/bd
# Hadoop version to install
ENV HADOOP_VERSION=3.5.0
ENV LOG_TAG="[BASE Hadoop_${HADOOP_VERSION}]:"
ENV REPOSITORY=https://dlcdn.apache.org/hadoop/common

# Change to root user
USER root

# Install necessary dependencies
RUN echo "$LOG_TAG Updating and installing basic packages" && \
    apt-get update && \
    apt-get install -y --no-install-recommends openjdk-17-jdk
    ↪  openjdk-17-jre python3 curl locales && \
    apt-get install -y nano && \
    apt-get install -y maven && \
    apt-get install -y dnsutils && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists
```

```
# Generate locales in English
RUN echo "$LOG_TAG Generating locales" && \
    locale-gen en_GB.UTF-8 && update-locale LANG=en_GB.UTF-8

# Make the directory for the Hadoop installation
RUN mkdir -p ${BASE_DIR}
# Change to that directory
WORKDIR ${BASE_DIR}

# Download the latest version of Hadoop to /opt/bd
RUN echo "$LOG_TAG Downloading Hadoop" && \
    curl -fLO
    ↪  "${REPOSITORY}/hadoop-${HADOOP_VERSION}/hadoop-${HADOOP_VERSION}.tar.gz"
    ↪  && \
    tar xzvf hadoop-${HADOOP_VERSION}.tar.gz && \
    ln -s hadoop-${HADOOP_VERSION} hadoop && \
    rm hadoop-${HADOOP_VERSION}.tar.gz

# Create the "hadoop" user group and
# Also create your user (USERNAME) that will be part of that group
RUN groupadd -r hadoop && \
    useradd -r -g hadoop -d ${BASE_DIR} -m -s /bin/bash ${USERNAME}

# Create the .bashrc and .bash_profile for USERNAME
# with the necessary environment variables
RUN echo 'source $HOME/.bashrc' >> .bash_profile
RUN echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >>
↪  /opt/bd/.bashrc
RUN echo 'export HADOOP_HOME=/opt/bd/hadoop' >> /opt/bd/.bashrc
RUN echo 'export PATH=$PATH:$HADOOP_HOME/bin:$JAVA_HOME/bin' >>
↪  /opt/bd/.bashrc
RUN chmod +x /opt/bd/.bashrc
RUN chmod +x /opt/bd/.bash_profile

# Create the directory for the log files
RUN mkdir /opt/bd/hadoop/logs

# Set permissions
RUN chown -R ${USERNAME}:hadoop ${BASE_DIR}

# Create a new user with lower privileges, only to run MapReduce jobs
RUN useradd -m -s /bin/bash luser
```

If you have an Apple MacBook. If you have a Mac with an ARM processor, you need to use the ARM image of Ubuntu. In addition, you need to modify the Dockerfile to download the ARM version of Hadoop. You can find it at this link under the name "aarch64". Also, you need to change the JAVA_HOME directory to `/usr/lib/jvm/java-17-openjdk-arm64`.

Once the Dockerfile has been completed, build the image with the following command (run it in a terminal on your computer, in the directory where the Dockerfile is located):

```
$ docker build -t hadoop-base .
```

We want to deploy a Hadoop cluster with the following characteristics:

- 1 master node as NameNode (HDFS) and ResourceManager (YARN)
- 4 worker nodes as DataNodes (HDFS) and NodeManagers (YARN)

Therefore, from this base image, we will create specific images for the NameNode/ResourceManager and DataNode/NodeManager machines. First of all, let's **create a network** that will be used by the containers (our nodes) to communicate with each other:

```
$ docker network create temporary-network
```

1.1 NameNode/ResourceManager

From the base image, we will create a new image for the NameNode/ResourceManager, but **first we need to edit some configuration files**. We run the container in interactive mode (*-ti*):

```
$ docker container run -ti --name namenode --network=temporary-network
 ↪  --hostname namenode --net-alias resourcemanager --expose 8000-10000
 ↪  -p 9870:9870 -p 8088:8088 hadoop-base /bin/bash
```

The command above will create a container named *namenode* with two hostnames: *namenode* and *resourcemanager*. The container is connected to the previously created network *temporary-network*. We need to be able to access the web interfaces of the NameNode and ResourceManager from the host. For this reason, we expose the ports 9870 and 8088 with the *–expose* option. Finally, we map the ports 9870 and 8088 of the container to the ports 9870 and 8088 of the host with the *-p* option. The image *hadoop-base* is the base image that we will use to create the new image. The last argument */bin/bash* is the command that we want to execute in the container. In this case, we want to start a bash shell to configure our NameNode and ResourceManager.

1.1.1 Creation of directories for NameNode data

We must specify the directory or directories in which the NameNode will store the HDFS metadata. In a real system, at least two directories should be used: one on the NameNode's local disk (preferably with several disks configured in RAID) and another on a remote location (for example, mounted by NFS).

In our case, **we will create a single directory owned by your user**. Your user is the user who will run the NameNode and ResourceManager daemons. To create the directory and change its ownership, we **run as root** the following commands in the container:

```
# mkdir -p /var/data/hdfs/namenode
# chown YOUR_USERNAME:hadoop /var/data/hdfs/namenode
```

1.1.2 Configuration of the NameNode/ResourceManager

We don't need root to edit the NameNode/ResourceManager configuration files. We must use your username.

```
# su - YOUR_USERNAME
```

All Hadoop daemons are mainly configured through four files, located in `$HADOOP_HOME/etc/hadoop/`, in which a large number of properties can be specified (see the official documentation for more information):

- **core-site.xml**: main configuration, default settings at http://hadoop.apache.org/docs/stable3/hadoop-project-dist/hadoop-common/core-default.xml
- **hdfs-site.xml**: HDFS configuration, default settings at http://hadoop.apache.org/docs/stable3/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml
- **yarn-site.xml**: YARN configuration, default settings at http://hadoop.apache.org/docs/stable3/hadoop-yarn/hadoop-yarn-common/yarn-default.xml
- **mapred-site.xml**: MapReduce configuration, default settings at http://hadoop.apache.org/docs/stable3/hadoop-mapreduce-client/hadoop-mapreduce-client-core/mapred-defau xml

We will use **nano** to edit these files through the terminal. Write the following parameters in the corresponding files:

core-site.xml - Hadoop main configuration

```
<configuration>

<property>
  <!-- Default filesystem name -->
  <!--  As we want to use HDFS we have to indicate it with hdfs://, -->
  <!-- and the server IP and port on which the NameNode will run -->
  <name>fs.defaultFS</name>
  <value>hdfs://namenode:9000/</value>
  <final>true</final>
</property>

<property>
  <!-- Directory for temporary storage (must have enough space) -->
  <name>hadoop.tmp.dir</name>
  <value>/var/tmp/hadoop-${user.name}</value>
  <final>true</final>
</property>

</configuration>
```

hdfs-site.xml - NameNode (HDFS) configuration

```
<configuration>

<property>
  <!-- Block replication factor -->
   <name>dfs.replication</name>
   <value>3</value>
   <final>true</final>
 </property>

 <property>
  <!-- Block size (default 128m) -->
   <name>dfs.blocksize</name>
   <value>64m</value>
   <final>true</final>
 </property>

 <property>
   <!-- Comma-separated list of directories where the NameNode
   stores metadata. -->
   <name>dfs.namenode.name.dir</name>
   <value>file:///var/data/hdfs/namenode</value>
   <final>true</final>
 </property>

 <property>
   <!-- address and port of the NameNode web interface -->
   <name>dfs.namenode.http-address</name>
   <value>namenode:9870</value>
   <final>true</final>
 </property>

</configuration>
```

yarn-site.xml - ResourceManager (YARN) configuration

```
<configuration>

<property>
  <!-- Hostname of the node running the ResourceManager daemon -->
  <name>yarn.resourcemanager.hostname</name>
  <value>resourcemanager</value>
  <final>true</final>
</property>

<property>
  <!-- Maximum number of vcores that an ApplicationMaster can request
  from the ResourceManager (default: 4).
  Larger requests throw an InvalidResourceRequestException -->
  <name>yarn.scheduler.maximum-allocation-vcores</name>
  <value>1</value>
  <final>true</final>
</property>

<property>
  <!-- Minimum memory (MB) that an ApplicationMaster can request
  from the ResourceManager (default: 1024).
  The memory allocated to a container must be a multiple of
  this amount -->
  <name>yarn.scheduler.minimum-allocation-mb</name>
  <value>128</value>
  <final>true</final>
</property>

<property>
  <!-- Maximum memory (MB) that an ApplicationMaster can request from
  the ResourceManager (default: 8192 MB).
  Larger requests throw an InvalidResourceRequestException.
  You can increase or decrease this value depending on
  how much memory you have -->
  <name>yarn.scheduler.maximum-allocation-mb</name>
  <value>2048</value>
  <final>true</final>
</property>

</configuration>
```

mapred-site.xml - MapReduce configuration

We are not going to use the Hadoop MapReduce implementation, but we need to configure it. Just copy the following configuration:

```
<configuration>

<property>
  <!-- Framework that performs the MapReduce -->
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
  <final>true</final>
</property>

<!-- MR ApplicationMaster (AM) configuration -->

<property>
  <!-- Location of the MR software for the AM -->
  <name>yarn.app.mapreduce.am.env</name>
  <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
</property>

<property>
  <!-- Maximum number of cores for the ApplicationMaster (default: 1) -->
  <name>yarn.app.mapreduce.am.resource.cpu-vcores</name>
  <value>1</value>
  <final>true</final>
</property>

<property>
  <!-- Memory needed by the MR ApplicationMaster (default: 1536) -->
  <name>yarn.app.mapreduce.am.resource.mb</name>
  <value>1536</value>
  <final>true</final>
</property>

<!-- MR maps and reductions configuration -->

<property>
    <!-- Ratio of heap size between container size for the JVMs
    (default: 0.8)-->
    <name>mapreduce.job.heap.memory-mb.ratio</name>
    <value>0.8</value>
    <final>true</final>
</property>
```

```
<!-- Maps -->
<property>
  <!-- MR software localisation for maps -->
  <name>mapreduce.map.env</name>
  <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
</property>

<property>
  <!-- Maximum number of cores for each map task (default: 1) -->
  <name>mapreduce.map.cpu.vcores</name>
  <value>1</value>
  <final>true</final>
</property>

<property>
  <!-- JVM options for maps -->
  <name>mapreduce.map.java.opts</name>
  <value>-Xmx1024M</value> <!-- Xmx defines the maximum size  -->
  <final>true</final>      <!-- of the Java heap -->
</property>

<property>
  <!-- Maximum memory (MB) per map (if -1 is obtained as
  heapSize / mapreduce.heap.memory-mb.ratio
  where heapSize is the value specified in -Xmx) -->
  <name>mapreduce.map.memory.mb</name>
  <value>-1</value>
  <final>true</final>
</property>

<!-- Reduces -->
<property>
  <!-- Location of the MR software for reducers -->
  <name>mapreduce.reduce.env</name>
  <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
</property>

<property>
  <!-- Maximum number of cores for each reduce task (default: 1) -->
  <name>mapreduce.reduce.cpu.vcores</name>
  <value>1</value>
  <final>true</final>
</property>

<property>
  <!-- JVM options for reducers -->
  <name>mapreduce.reduce.java.opts</name>
  <value>-Xmx1024M</value> <!-- Xmx defines the maximum size  -->
  <final>true</final>      <!-- of the Java heap -->
</property>
```

```
  <property>
    <!-- Maximum memory (MB) per reduce (if -1 is obtained as
    heapSize / mapreduce.heap.memory-mb.ratio
    where heapSize is the value specified in -Xmx) -->
    <name>mapreduce.reduce.memory.mb</name>
    <value>-1</value>
    <final>true</final>
  </property>
</configuration>
```

1.1.3 Formatting the HDFS filesystem

The first time we start the NameNode, we must format the HDFS filesystem to create the directory structure and the initial metadata. To do this, we must run the following command with your username:

```
$ hdfs namenode -format
```

At the end of the formatting process, if everything went well, the following message, among others, should appear on the terminal:

`Storage directory /var/data/hdfs/namenode has been successfully formatted.`

Check that the directory `/var/data/hdfs/namenode` exists (e.g. with `ls`). Also, check that a directory for logs has been created in `$HADOOP_HOME/logs`.

1.1.4 Starting the NameNode/ResourceManager

Now we are ready to run our first node. To start the NameNode/ResourceManager daemon, we must run the following command with your username:

1) Start the HDFS NameNode daemon:

```
$ hdfs --daemon start namenode
```

Look at the files created in the log directory (`$HADOOP_HOME/logs`) to check that everything went well and that no errors appeared (e.g. using `tail`). Run the `jps` command to verify that this daemon is running in the Java Virtual Machine (JVM).

2) Start the YARN ResourceManager daemon:

```
$ yarn --daemon start resourcemanager
```

Again, look at the files created in the log directory (`$HADOOP_HOME/logs`) to check that everything went well and that no errors appeared. Run one more time the `jps` command to see that the daemon is running in the JVM.

1.1.5 Checking the NameNode/ResourceManager web interfaces

The different Hadoop services offer a web interface for monitoring. Open a browser on your PC and check that the following URLs are accessible:

- http://localhost:9870 HDFS web interface
- http://localhost:8088 YARN web interface

1.1.6 Stopping the NameNode/ResourceManager

To stop the daemons, run the above commands in reverse order, changing start to stop:

```
$ yarn --daemon stop resourcemanager
$ hdfs --daemon stop namenode
```

1.1.7 Automation of the deployment of the Hadoop cluster

We have seen how to configure and launch the NameNode/ResourceManager manually. Now we are going to automate the daemon startup process. It will be useful when we want to reboot the cluster.

Therefore, we will create a script that will be executed when the container starts. This script will start the NameNode/ResourceManager daemons. **As root**, in the root directory (`cd /`), create the file `/start.sh` with the following content:

```
#!/bin/bash
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/bd/hadoop

# Start the NameNode and ResourceManager daemons
su YOUR_USERNAME -c "$HADOOP_HOME/bin/hdfs --daemon start namenode"
su YOUR_USERNAME -c "$HADOOP_HOME/bin/yarn --daemon start
↪  resourcemanager"

# Loop to keep the container running
while true; do sleep 10000; done
```

Then, make the file executable:

```
# chmod +x /start.sh
```

1.1.8 Creation of the image for the NameNode/ResourceManager

We have now configured our first node, the master node that will run the NameNode and ResourceManager daemons. Now, using this container with the NameNode and ResourceMan-

ager perfectly configured, we are going to create an image so that we can create new containers without having to configure them again. Logout from the container and in your computer, create the image:

```
docker container commit namenode namenode-image
```

Check that the image has been created:

```
docker images
```

If all went well, remove the container:

```
docker container rm namenode
```

1.2 DataNode/NodeManager

From the base image (*hadoop-base*), we will create a new image for the DataNode(s)/NodeManager(s). We will follow similar steps as for the NameNode/ResourceManager, but the configuration is different. Run the container in interactive mode (*-ti*):

```
$ docker container run -ti --name datanode --network=temporary-network
 ↪  --hostname datanode --expose 8000-10000 --expose 50000-50200
 ↪  hadoop-base /bin/bash
```

In this case, we have exposed different ports to allow the NodeManager/ResourceManager to communicate with each DataNode/DataManager.

1.2.1 Creation of directories for DataNode data

We must specify the directory or directories in which the DataNodes will store the HDFS blocks. Ideally, you should have a node with several disks and create a directory on each of the local disks to improve access performance. RAID is not recommended, as HDFS guarantees redundancy.

In our case, we will create a single directory, executing in the docker container, as root, these commands:

```
# mkdir -p /var/data/hdfs/datanode
# chown YOUR_USERNAME:hadoop /var/data/hdfs/datanode
```

Again, we have created a directory owned by your user.

1.2.2 Configuration of the DataNode/NodeManager

For our DataNode/NodeManager, with your user (`su - YOUR_USERNAME`), edit the following configuration files in `$HADOOP_HOME/etc/hadoop/`:

core-site.xml - Main configuration, same as in the NameNode.

mapred-site.xml - MapReduce configuration, same as in the NameNode.

hdfs-site.xml - HDFS Configuration

We want to limit the space that each DataNode can use to store HDFS blocks.

1. Run the command `df -h` to see the available space in the root partition (`/`) of the container. Write it down.
2. We want to allocate only 200 GB per DataNode for HDFS storage (which must be **less than the total available space**). The remainder of the partition must be reserved for other uses (i.e., total partition size − 200 GB). For example, with a 1,000 GB partition, we reserve $800GB \cdot \frac{1024MB}{1GB} \cdot \frac{1024KB}{1MB} \cdot \frac{1024B}{1KB} = 858993459200B$. Adjust this calculation according to the available space on your partition.
3. That value is what we have to write in the parameter `dfs.datanode.du.reserved` of the file `hdfs-site.xml`.

```
<configuration>

<property>
  <!-- Comma-separated list of directories where
  the DataNodes storethe HDFS blocks -->
  <name>dfs.datanode.data.dir</name>
  <value>file:///var/data/hdfs/datanode</value>
  <final>true</final>
</property>

<property>
  <!-- Reservation of space in bytes that the DataNode
  cannot use to store HDFS blocks. -->
  <name>dfs.datanode.du.reserved</name>
  <value>WRITE HERE YOUR VALUE</value>
</property>

</configuration>
```

yarn-site.xml - YARN Configuration

```
<configuration>

<property>
  <!-- Hostname of the machine running the ResourceManager daemon -->
  <name>yarn.resourcemanager.hostname</name>
  <value>resourcemanager</value>
  <final>true</final>
</property>

<property>
  <!-- Enable auto-detection of node capacities (memory and CPU) -->
  <name>yarn.nodemanager.resource.detect-hardware-capabilities</name>
  <value>true</value>
  <final>true</final>
</property>

<property>
  <!-- Number of vcores that can be allocated for YARN containers
  If -1, it is automatically detected (if auto-detection is enabled) -->
  <name>yarn.nodemanager.resource.cpu-vcores</name>
  <value>1</value>
  <final>true</final>
</property>

<property>
  <!-- MB of physical RAM that can be reserved for YARN
  containers (default: 8192). Must be less than the physical
  RAM, to allow other services to run.
  If -1, it is auto-detected (if auto-detect is enabled).
  Can be changed to a fixed value, e.g. 3072 (3 GB) -->
  <name>yarn.nodemanager.resource.memory-mb</name>
  <value>1536</value>
  <final>true</final>
</property>

<property>
  <!-- Disable checking of virtual memory usage limits -->
  <name>yarn.nodemanager.vmem-check-enabled</name>
  <value>false</value>
  <final>true</final>
</property>
```

```
<property>
  <!-- Tell the NodeManagers that they have to implement
  the MapReduce shuffle service -->
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
  <final>true</final>
</property>

<property>
  <!-- Class that implements the MapReduce shuffle service -->
  <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  <final>true</final>
</property>

</configuration>
```

1.2.3 Starting the DataNode/NodeManager

Now we are ready to run our second node, the DataNode/NodeManager. To start the DataNode/NodeManager daemon, we must run the following command with your username:

1) Start the HDFS DataNode daemon:

```
$ hdfs --daemon start datanode
```

Look at the files created in the log directory (`$HADOOP_HOME/logs`) to check that everything went well and that no errors appeared. `Retrying connect to server: namenode:9000` in the logs is normal, as the NameNode is not running yet (we stopped it before). Run the `jps` command to verify that this daemon is running in the Java Virtual Machine (JVM).

2) Start the YARN NodeManager daemon:

```
$ yarn --daemon start nodemanager
```

Again, look at the files created in the log directory (`$HADOOP_HOME/logs`) to check that everything went well and that no errors appeared. `Retrying connect to server: resourcemanager:8031` in the logs is normal, as the ResourceManager is not running yet (we stopped it before). Run one more time the `jps` command to see that the daemon is running in the JVM.

1.2.4 Stopping the DataNode/NodeManager

To stop the daemons, run the above commands in reverse order, changing start to stop:

```
$ yarn --daemon stop nodemanager
$ hdfs --daemon stop datanode
```

1.2.5 Automation of the deployment of the Hadoop cluster

We have seen how to configure and launch the DataNode/NodeManager manually. Now we are going to automate the daemon startup process. It will be useful when we want to reboot the cluster or when we want to create new DataNode(s)/NodeManager(s).

As we did for the NameNode/ResourceManager, we will create a script that will be executed when the container starts. This script will start the DataNode/NodeManager daemons. **As root**, in the root directory (`cd /`), create the file `/start.sh` with the following content:

```
#!/bin/bash
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/bd/hadoop

# Start the DataNode and NodeManager daemons
su YOUR_USERNAME -c "$HADOOP_HOME/bin/hdfs --daemon start datanode"
su YOUR_USERNAME -c "$HADOOP_HOME/bin/yarn --daemon start nodemanager"

# Loop to keep the container running
while true; do sleep 10000; done
```

Remember to make the file executable:

```
# chmod +x /start.sh
```

1.2.6 Creation of the image for the DataNode/NodeManager

We have now configured our second node, the DataNode/NodeManager. Now we are going to create an image from this container so that we can create new DataNode(s)/NodeManager(s) with the same configuration. Logout from the container and in your computer, create the image as we did for the NameNode/ResourceManager:

```
docker container commit datanode datanode-image
```

Check that the image has been created, and remove the container and the temporary network:

```
docker container rm datanode
docker network rm temporary-network
```

1.3 Deployment of the Hadoop cluster using Docker Compose

Now we have the images for the NameNode/ResourceManager and the DataNode/NodeManager. We are going to use Docker Compose to deploy the cluster. Docker Compose is a tool for defining and running multi-container Docker applications. With Compose, we use a YAML file to configure our application's services. Then, with a single command, we create and start all the services configured in that file.

First, we need to create a directory for the configuration file. In your computer, create a directory named `cluster-compose` and enter it. Then, create the file `docker-compose.yml`. **In the next page, you can see an example** for a Hadoop cluster with 1 NameNode/ResourceManager and 1 DataNode/NodeManager.

For the `namenode` service/machine, we have defined that the namenode-image created earlier must be used. We have also defined the name of the container (*namenode*), the network to which it belongs (*hadoop-cluster*), the hostname (*namenode*), and also the alias *resourcemanager*. We have also defined the number of CPUs and the memory limit. In a real environment, the node would not be limited to only 1 core and 3GB of memory. **You can adjust the amount of RAM according to your computer.** We have also exposed the ports 9870 and 8088, mapped them to the host, and defined the volumes that we want to mount. Docker volumes are the recommended way to persist data generated and modified within Docker containers. This way, in case we have to delete the containers, we will not lose the information stored in those directories. In this case, we have defined two volumes: one for the HDFS metadata and another for the configuration files. Finally, we have defined the command that will be executed when the container starts, which is the script `/start.sh` to start the NameNode/ResourceManager daemons.

For the `datanode1` service/machine, a similar configuration has been defined. The only difference is that we used the image *datanode-image*, that we have not defined the alias *resourcemanager*, and that we have exposed a different range of ports to allow the communication with the NodeManager/ResourceManager. We have also defined the volume for the HDFS blocks. You must **complete this configuration for the rest of the DataNodes that we want to deploy (4 in total)**.

```yaml
services:
  namenode:
    image: namenode-image
    container_name: namenode
    networks:
      hadoop-cluster:
        aliases:
          - resourcemanager
    hostname: namenode
    deploy:
      resources:
        limits:
          cpus: '1'
          memory: 3072m
    expose:
      - "8000-10000"
    ports:
      - "9870:9870"
      - "8088:8088"
    volumes:
      - namenode:/var/data/hdfs/namenode
      - configNamenode:/opt/bd/hadoop/etc/hadoop/
    command: /start.sh

  datanode1:
    image: datanode-image
    container_name: datanode1
    networks:
      - hadoop-cluster
    hostname: datanode1
    deploy:
      resources:
        limits:
          cpus: '1'
          memory: 3072m
    expose:
      - "8000-10000"
      - "50000-50200"
    volumes:
      - datanode1:/var/data/hdfs/datanode
    command: /start.sh

    # Add three more DataNodes here

networks:
  hadoop-cluster:

volumes:
  namenode:
  configNamenode:
  datanode1:
  # Add three more DataNodes here
```

To deploy the cluster, run the following command in the directory where the `docker-compose.yml` file is located:

```
docker compose up -d
```

Check that the containers have been created and are running:

```
docker container ps
```

Connect to the NameNode/ResourceManager container:

```
docker container exec -ti namenode /bin/bash
```

With your *username*, check that everything is working correctly:

```
$ hdfs dfsadmin -report
$ yarn node -list
```

Also, go to the web interfaces and check that nodes are listed there.

For security reasons, **remember to stop the cluster when you shutdown your computer**. To stop the cluster, run `docker compose stop` in the directory where the `docker-compose.yml` file is located. To restart it, run `docker compose start`. To delete the cluster, run `docker compose down --volumes`.

1.4 Basic use of the HDFS filesystem

We already have our Hadoop cluster running, i.e., a distributed filesystem (HDFS) and a resource manager (YARN). Let's see how to use the HDFS filesystem.

1.4.1 Creating directories in HDFS

We want to create a different directory for each user, like `/home/USER/` in Linux systems. Specifically, we will create the directory `/user/` in HDFS, and within it, a directory for each user. To do this, connect to the NameNode/ResourceManager container and, with your *username*, run the following commands:

```
$ hdfs dfs -mkdir -p /user/YOUR_USERNAME
$ hdfs dfs -mkdir -p /user/luser
$ hdfs dfs -chown luser /user/luser
$ hdfs dfs -ls /user
```

Note that both directories are owned by *YOUR_USERNAME*. This is because your user is the user who runs the NameNode/ResourceManager daemon, and therefore, the user who has the permissions to create directories in HDFS. However, *luser* is the user who will run the MapReduce jobs, and therefore, this user will also need to create files in HDFS. For this reason, we have changed the ownership of the `/user/luser/` directory to *luser*.

We must also create the directory for temporary files:

```
$ hdfs dfs -mkdir -p /tmp/hadoop-yarn/staging
$ hdfs dfs -chmod -R 1777 /tmp
```

Check that the directories have been created correctly:

```
$ hdfs dfs -ls /
```

1.4.2 Uploading files to HDFS

As *luser* is the user who will run the MapReduce jobs, we will upload the files as *luser*. To do this, as *luser* (`su - luser` as root), add the following lines to the `.bashrc` file:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/bd/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$JAVA_HOME/bin
```

Then, execute the .bashrc file to load the new environment variables:

```
$ . ~/.bashrc
```

Check that can access the HDFS filesystem:

```
$ hdfs dfs -ls
```

This command shouldn't show any output because we haven't uploaded any files yet. The directory `/user/luser/` is empty.

Now, we are going to upload a file to HDFS. Download this file to your computer. Copy the file to the container:

```
docker container cp ~/Downloads/books.tar namenode:/tmp
```

In the NameNode, extract the .tar file and copy the files to HDFS:

```
$ cd /tmp; tar xvf books.tar
$ hdfs dfs -put books .
```

Check that the files have been uploaded correctly:

```
$ hdfs dfs -ls books
```

Once we have uploaded the files, we can delete the `/tmp/books/` directory and the `books.tar` file.

```
$ rm -rf /tmp/books
# rm /tmp/books.tar
```

Finally, check in the HDFS web interface that the files have been uploaded correctly. See where the blocks of the **random_words.txt.bz2** file are located. Check that it has 3 replicas.

- Open the HDFS web interface (http://localhost:9870).
- Utilities > Browse the file systems

Looking at the web interface, **answer these questions**: In how many blocks is the random_words1.tar file divided? In which DataNodes is block 2 of this file located?

1.5 Test a simple MapReduce job

1.5.1 Computing exact digits of π

Hadoop comes with some examples that we can use to test our cluster. They are located in the `$HADOOP_HOME/share/hadoop/mapreduce/` directory. A simple example is the distributed computation of the exact digits of π. See Hadoop documentation for more information about the implementation.

In the NameNode/ResourceManager container, with your username, run the following commands:

```
$ export MAPRED_EXAMPLES=$HADOOP_HOME/share/hadoop/mapreduce
$ yarn jar $MAPRED_EXAMPLES/hadoop-mapreduce-examples-*.jar pi 16 1000
```

Check in the YARN web interface the status of the job. Once it is finished, check it has been executed correctly.

1.5.2 WordCount

Another example is WordCount. It counts the number of occurrences of each word in a set of files. Let's count the number of occurrences of each word in the books that we have uploaded to HDFS (books.tar).

Download the Java code from here and copy it to the NameNode/ResourceManager container:

```
docker container cp ~/Downloads/wordcount.tgz namenode:/home/luser
```

In the NameNode/ResourceManager container, as *luser*, extract the .tgz file and compile the code:

```
$ cd; tar xvzf wordcount.tgz
$ cd wordcount
$ mvn package
```

Execute the WordCount example with the `yarn` command:

```
$ yarn jar target/wordcount*.jar books/p* wordcount-out
```

Check in the YARN web interface the status of the job. Once it is finished, check it has been executed correctly.

Then, copy the output to the local filesystem:

```
$ hdfs dfs -get wordcount-out
```

Check the output files in the local filesystem with, for example, the `cat` command. **How many output files are there? Why?**

2 Assignment

You have to complete four tasks related to the deployment of a Hadoop cluster using Docker. Pay attention to the instructions. You will have to submit a PDF report with the steps followed and the results obtained. The report has to include screenshots of the web interfaces and the commands executed. For each task, the screenshots to be included in the report will be specified.

2.1 Task 1: Add a Backup node and a TimeLineServer to the cluster

2.1.1 Backup node

As seen in the theory, the Backup node performs the same checkpoint function as the Secondary NameNode, but it always has the up-to-date state of the namespace. It receives real-time streaming of edits from the NameNode. If the NameNode fails, the Backup node can be used to restart the cluster. In this task, you have to add a Backup node to the cluster.

First of all, run the cluster, connect to the NameNode/ResourceManager container, and **take a screenshot** in which we can see the metadata files of the HDFS filesystem. You can see the metadata files in the `current` directory.

To add a Backup server, we start again from the hadoop-base image. We will create a new image for the Backup node. Check the name of the network with the command `docker network ls`. Run the container in interactive mode (*-ti*) (edit the network name if necessary):

```
docker container run -ti --name backupnode
 ↪  --network=cluster-compose_hadoop-cluster --hostname backupnode
 ↪  --cpus=1 --memory=3072m --expose 50100 -p 50105:50105 hadoop-base
 ↪  /bin/bash
```

In this case, we have exposed different ports to allow the NameNode/ResourceManager to communicate with the Backup node. We have also mapped the port 50105 to the host. This port is used to access the Backup node web interface.

Then, do the following:

1. Create a directory in which the Backup node will store the metadata backup. Remember that **the directory must be owned by your user**. Inside this directory, create the `dfs/name` directories.

2. With your username, edit the `core-site.xml` file. Add the following properties:
   - **fs.defaultFS**: Default filesystem URI. Give it the value hdfs://namenode:9000
   - **hadoop.tmp.dir**: Directory where Hadoop will store the metadata backup. Give it the value of the directory created in the previous step (without the `dfs/name`).
3. With your username, edit the `hdfs-site.xml` file. Add the following properties:
   - **dfs.namenode.backup.address**: Address and port of the Backup node. Give it the value backupnode:50100
   - **dfs.namenode.backup.http-address**: Address and port of the Backup node web interface. Give it the value backupnode:50105
4. Start the Backup node daemon. With your username, run the following command:

```
$ hdfs namenode -backup
```

5. Check the backup directory to see what files have been created. Compare them with the NameNode metadata screenshot taken at the beginning. What differences do you see? **Take a screenshot** of the metadata files in the Backup node.
6. Look in the backup service messages for information indicating that a checkpoint has been made.

Once you have taken the screenshots for the report, create the image for the Backup node (call it `backupnode-image`) and remove the container. Next, edit the `docker-compose.yml` file to add the Backup node to the cluster. It has to be added taking into account **the same parameters as the top docker run**. Use a volume to persist the metadata files of the HDFS filesystem as we did with the other nodes. As a command to start the container, use the following:

```
command: su YOUR_USERNAME -c "sleep 30 &&
↪  JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 /opt/bd/hadoop/bin/hdfs
↪  namenode -backup"
```

Finally, run the new node using `docker compose up -d` and check that the Backup node is working correctly by accessing its web interface and checking the logs with the `docker container logs backupnode` command.

Information for the report:

- Commands executed to complete the task.
- Screenshot of the metadata files in the NameNode before the Backup node was created.
- Screenshot of the metadata files in the NameNode after the Backup node was created.
- Screenshot of the metadata files in the Backup node. Compare the files in the three screenshots and explain the differences based on what we have learned in the theory class. What process has been carried out?
- Screenshot showing the messages generated by the backup service, highlighting those showing how the checkpoint is made.
- Screenshot of the backup node web interface with the checkpoint process finished.

If your computer has limited resources, you can delete the Backup node to free up resources.

2.1.2 TimeLineServer

The YARN TimelineServer maintains a history and provides metrics of applications executed in YARN. It provides both generic information about applications (the list of containers run under each application-attempt, queue-name, user information, a list of application-attempts that ran for an application, etc.) and specific information to the application's particular framework (e.g. the MapReduce framework can publish information about the number of maps and reduces, or other counters). The information is accessible through a web interface or via a REST API.

The Timeline Server can run on a different node or be placed on the same node as the ResourceManager. More information about the TimeLineServer is available at the documentation.

To add the TimeLineServer, you have to follow the following steps (with the cluster running):

- Connect to the ResourceManager container and, with your username, stop the ResourceManager daemon.
- In the ResourceManager edit the `yarn-site.xml` file. Add the following properties:
  - **yarn.timeline-service.hostname**: Hostname of the TimeLineServer machine. Give it the value *timelineserver* (this will be the name of the Docker container that will run the service).
  - **yarn.timeline-service.enabled**: Indicates whether the timeline service is active or not. Set it to true.
  - **yarn.system-metrics-publisher.enabled**: Indicates the ResourceManager to publish YARN metrics to the timeline server. Set to true.
- Start again the ResourceManager daemon.
- Add a new service to the `docker-compose.yml` file.
  - Name and hostname: timelineserver
  - Expose: 10200
  - Ports: 8188:8188
  - Image: hadoop-base
  - Command:

```
su YOUR_USERNAME -c
 ↪  "JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
 ↪  /opt/bd/hadoop/bin/yarn --daemon start timelineserver &&
 ↪  while true; do sleep 10000; done"
```

  - Rest of parameters: same as the other workernodes.
- In the ResourceManager, execute a MapReduce job (for example, WordCount).
- Check that the information about the execution is available in the TimeLineServer web interface http://localhost:8188.

If your computer has limited resources, you can delete the TimeLineServer to free up resources after taking the screenshots for the report.

Information for the report:

- Commands executed to complete the task.
- Screenshot of the TimeLineServer web interface showing information on the execution of one or more jobs.

2.2 Task 2: Decommissioning a DataNode/NodeManager

2.2.1 Creation of included and excluded node files

Although it is not strictly necessary to add or remove nodes from the cluster, it is convenient to have a list in which we can indicate the nodes that are part of the cluster and those that are not. This is useful when we want to do a scheduled shutdown of a DataNode/NodeManager or when we want to add a new node to the cluster. To do this, do the following in the NameNode (with your username):

1. Stop the NameNode and ResourceManager daemons.
2. Create four files: `${HADOOP_HOME}/etc/hadoop/dfs.include`, `${HADOOP_HOME}/etc/hadoop/dfs.exclude`, `${HADOOP_HOME}/etc/hadoop/yarn.include`, and `${HADOOP_HOME}/etc/hadoop/yarn.exclude`
3. In the `dfs.include` and `yarn.include` files, add the name of the DataNodes that you want to include in the cluster (one name per line), i.e., datanode1, datanode2, datanode3, and datanode4. Leave the dfs.exclude and yarn.exclude files empty.
4. In the configuration file **hdfs-site.xml**, add two properties:
   - **dfs.hosts**: name of the file with a list of hosts that can act as DataNodes; if the file is empty, any node is allowed. Give as value the full path to the dfs.include file.
   - **dfs.hosts.exclude**: name of the file with a list of hosts that cannot act as DataNodes; if the file is empty, none are excluded. Give as value, the complete path to the dfs.exclude file.
5. In the configuration file **yarn-site.xml**, add two properties:
   - **yarn.resourcemanager.nodes.include-path**: name of the file with a list of hosts that can act as NodeManagers; if the file is empty, any node is allowed. Give as value the full path to the yarn.include file.
   - **yarn.resourcemanager.nodes.exclude-path**: name of the file with a list of hosts that cannot act as NodeManagers; if the file is empty, none are excluded. Give as value, the complete path to the yarn.exclude.
6. Start the NameNode and ResourceManager daemons.
7. Check in the log files that datanode1, datanode2, datanode3, and datanode4 have been included. Commands such as `cat` and `grep` can be useful.

2.2.2 Decommissioning a DataNode/NodeManager

In theory, the shutdown of a DataNode/NodeManager can be done directly and does not affect the cluster. However, if we want to do a scheduled shutdown of a DataNode/NodeManager it is preferable to notify the NameNode/ResourceManager beforehand.

Follow the steps below to remove, for example, datanode4.

1. First of all, take a screenshot of a block that is stored on the DataNode that we are going to remove.
2. With your username, write the name of the node(s) to be removed in the dfs.exclude and yarn.exclude file and run

   ```
   $ hdfs dfsadmin -refreshNodes
   $ yarn rmadmin -refreshNodes
   ```

3. Check that after a while, using the web interface and the commands `hdfs dfsadmin -report` and `yarn node -list`, the excluded node(s) appear(s) "Decommissioned" in HDFS and YARN. Take screenshots.
4. We can now stop the daemons on the decommissioned node and stop the associated container. As we don't want to include it in the cluster again, we can delete its Docker container. Remove the node from the docker-compose.yml file and run `docker compose up -d --remove-orphans` to update the cluster.
5. Remove the node(s) from the include and exclude file(s) and run again:

   ```
   $ hdfs dfsadmin -refreshNodes
   $ yarn rmadmin -refreshNodes
   ```

6. Check what happened now with those blocks that were stored on the decommissioned node. Take a screenshot of one of those cases.

Information for the report:

- Commands executed to complete the task.
- Screenshot of the HDFS web interface showing that datanode4 is decommissioned
- Screenshot of the YARN web interface showing that datanode4 no longer appears in the list of available nodes and that the number 1 appears in the decommissioned node counter.
- Screenshot of one of these blocks that were stored on datanode4 before it was removed. Comment on what happened.

2.3 Task 3: Adding a new DataNode/NodeManager

We will now add **two new DataNodes/NodeManagers to the cluster**. First, to see how the cluster behaves when new nodes are added, let's copy more data to our HDFS. Check with the `hdfs dfsadmin -report` command how is the space used in the cluster. It must be higher

than 8%. If it is not, copy more data to HDFS. To do it quickly, with your username, **extract the books.tar file again and copy the files to the YOUR_USERNAME directory in the HDFS filesystem**, as we did before with *luser*. Repeat the copy process until the space used in the cluster is higher than 8%. Then, follow the steps below:

1. Edit the docker-compose.yml file to add the two new nodes (datanode5 and datanode6) to the cluster. Use the same parameters as the other DataNodes/NodeManagers.
2. In the NameNode with your username, add the names of two new nodes in the dfs.include and yarn.include files.
3. Run up the new nodes using `docker compose up -d`.
4. Refresh nodes of the NameNode and the ResourceManager.
5. Check using (in the NameNode with your username) the hdfs commands `hdfs dfsadmin -report` and `yarn node -list` that the new nodes have been added to HDFS and YARN. You can also check the NameNode and YARN web interfaces. If they do not appear, refresh the nodes again.
6. Write down how much space, in bytes and blocks, the two new DataNodes store.

The new nodes are initially empty (they have no HDFS data), so the cluster will be unbalanced. You can force the balancing by running, in the NameNode:

```
$ hdfs balancer -threshold 1
```

Now check how much space, in bytes and blocks, the two new DataNodes store.

Information for the report:

- Commands executed to complete the task.
- Screenshot of the NameNode and ResourceManager log files showing that the new nodes have been added to the cluster.
- Screenshots of the outputs of the `dfsadmin -report` and `yarn node -list`.
- Screenshots of the NameNode web interface showing the number of blocks and space used for each DataNode before and after balancing.
- Screenshot of the output of `hdfs balancer -threshold 1`. Also, highlight those messages that indicate the number of bytes and blocks that have been moved.

2.4 Task 4: Rack Awareness

For best performance, it is important to configure Hadoop to be aware of our network topology. By default, Hadoop considers all DataNodes/NodeManagers to be the same and located in a single rack, which is identified as `/default-rack`.

For clusters with more than one rack, we must indicate to Hadoop which rack each node is in, to improve efficiency and reliability. For example, HDFS will try to store replicas of the same block in different racks.

Figure 1 shows a typical two-level network architecture for a Hadoop cluster. This architecture can be seen as a tree, where the root is the switch that connects both racks. The leaves are the

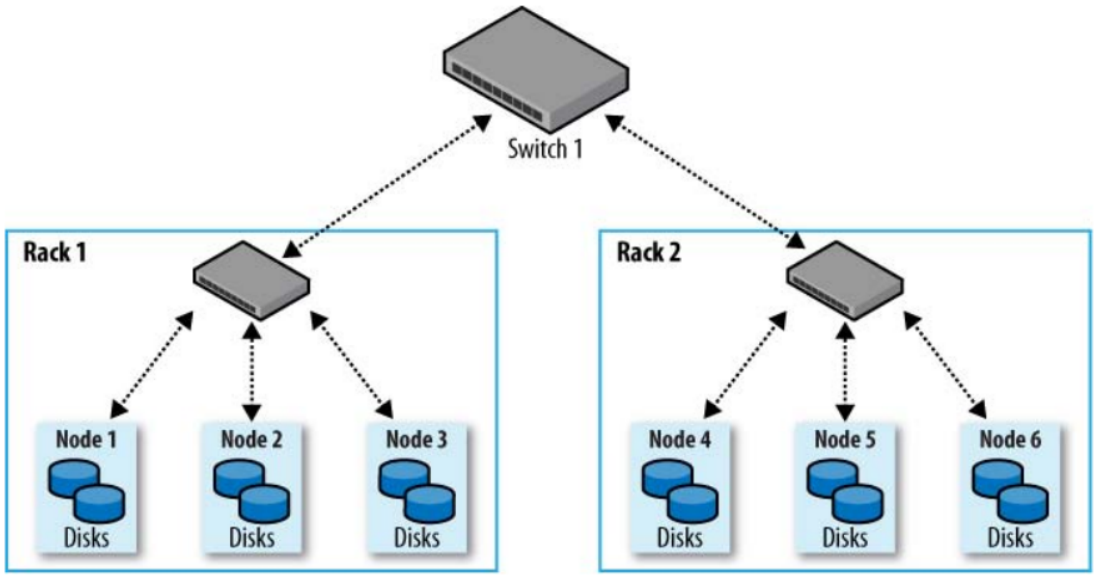


Figure 1: Typical two-level network architecture for a Hadoop cluster. *Source: T. White. Hadoop: The Definitive Guide.*

nodes of the cluster. To tell Hadoop this topology, it is necessary to use a script that maps the node names to the rack they are in.

In our case, let's assume that we have 3 racks (rack1, rack2, and rack3) and that we have two nodes in each rack. Do the following in the NameNode (with your username):

1. Run the command `hdfs dfsadmin -printTopology` to see the current topology.
2. Stop the NameNode daemon.
3. Create a file `$HADOOP_HOME/etc/hadoop/topology.data` that has in each line the hostname of one of the DataNodes and the rack where it is, as in this example (adapt it to our case):

```
datanode1     /rack1
datanode2     /rack1
datanode3     /rack2
datanode5     /rack3
datanode6     /rack3
```

4. Create a bash script `$HADOOP_HOME/etc/hadoop/topology.script` like the following (based on source). Give it execute permissions (`chmod +x topology.script`).

```
#!/bin/bash
# This script is called by Hadoop to get the network topology
# of the nodes. It receives as arguments the IPs of the nodes
# and must return the topology (rack) of each node.

HADOOP_CONF=$HADOOP_HOME/etc/hadoop
while [ $# -gt 0 ] ; do
  # Get the node hostname from its IP
  nodeArg=$(host "$1" | awk '/domain name pointer/ {print $5}' | sed 's/\.$//' )
  nodeArg=${nodeArg%%.*}
  # Read the topology from the topology.data file
  exec< ${HADOOP_CONF}/topology.data
  result=""
  while read line ; do
    ar=( $line )
    if [ "${ar[0]}" = "$nodeArg" ] ; then
      result="${ar[1]}"
    fi
  done
  shift
  if [ -z "$result" ] ; then
    echo -n "/default-rack "
  else
    echo -n "$result "
  fi
done
```

5. Define in the **core-site.xml** file the **net.topology.script.file.name** property and give it **the full path to the script** as value.
6. Start the NameNode daemon. Run the command `hdfs dfsadmin -printTopology` again to check that the topology has been updated.

Information for the report:

- Commands executed to complete the task.
- Screenshot showing the output of the `hdfs dfsadmin -printTopology` command showing the new topology.
- The `Dockerfile` and the last version of the `docker-compose.yml` file (attached in a zip file along with the report).

AI Usage Statement

The use of AI systems must be explicitly declared at the time of submission. In the case you have used AI systems, you must include a section (preferably at the end of the report) with the title *AI Usage Statement* in which you describe how you have used AI systems to help you complete the practical. You can use the following template:

Debugging and configuration assistance

Tools:	Claude Code and GitHub Copilot.
Purpose:	To help interpret daemon logs and debug Docker/Hadoop configuration issues encountered while deploying the cluster, and to review the syntax of the XML configuration files and Bash scripts involved in the tasks.
Key prompt:	*After running* `docker compose up -d`*, the datanode container exits immediately and the log shows* `Retrying connect to server: namenode:9000`*. Here are my* `core-site.xml`*,* `hdfs-site.xml` *and* `/start.sh`*. What could be causing it and how can I verify the fix?"*
Integration:	The suggestions were used only as hypotheses to guide debugging. Each proposed change was applied individually and verified on my own cluster using the commands documented in the practical (`hdfs dfsadmin -report`, `yarn node -list`, `jps`) and the web interfaces; the final configuration is the result of my own testing.
Ethics:	AI was used strictly as a debugging aid, not to generate the report or the screenshots, which correspond to executions on my own cluster. I take full responsibility for all commands, configuration files and code submitted, and I verified every AI-suggested change before including it, in line with the course's AI-use policy.